

DeepLift: Not Just A Black Box: Learning Important Features Through Propagating Activation Differences

Le-Van Thai

May 2026

1 DeepLIFT versions

- Paper ban đầu: dùng difference-from-reference để propagate contribution scores.
- Version mới (2017):
 - cải thiện cách xử lý nonlinearities,
 - tách positive/negative contributions $\rightarrow$ một feature làm tăng prediction, feature khác làm giảm prediction, nếu cộng chung có thể triệt tiêu nhau. $\rightarrow$ giữ riêng thì explanation rõ hơn.
 - Version cũ: natural images, genomic data.
 - Version mới: MNIST, simulated genomic data.
- Core idea vẫn giữ nguyên:

$$\delta_n = A_n - A_n^0$$

2 Introduction

- **Interpretability:** Neural networks thường là “black box”, gây cản trở (barrier to adoption) trong các lĩnh vực cần giải thích dự đoán. Việc hiểu rõ feature giúp tăng độ tin cậy và hỗ trợ khám phá khoa học.
- **Gradient-based Methods:** Bao gồm Saliency maps, Deconvolutional networks và Guided backpropagation. Feature importance được tính qua $x \cdot \frac{\partial y}{\partial x}$, dựa trên:
 - *Gradient* ($\frac{\partial y}{\partial x}$): Đo độ nhạy của output với input.
 - *Gradient $\times$ Input*: Cải thiện saliency maps nhờ xét thêm độ lớn input, tương đương Taylor approximation bậc nhất: $f(x) - f(0) \approx x f'(x)$.

- Ví dụ: Với hàm tuyến tính $f(x) = 3x$ tại $x = 2$, $xf'(x) = 6$ (chính xác). Với hàm phi tuyến $f(x) = x^2$ tại $x = 2$, $xf'(x) = 8$ trong khi thực tế $f(2) - f(0) = 4$ (chỉ là xấp xỉ).
- **Hạn chế của Gradient:** Vấn đề cốt lõi là *gradient nhỏ/zero* $\neq$ đóng góp nhỏ.
 - *ReLU*: Khi neuron không kích hoạt ($x < 0$), gradient $\frac{\partial y}{\partial x} = 0$ dù feature vẫn có thể chứa thông tin.
 - *Saturation*: Các hàm Sigmoid/Tanh khi input cực đại/cực tiểu sẽ có $\frac{\partial y}{\partial x} \approx 0$, làm mất dấu vết ảnh hưởng của feature.
- **DeepLIFT:** Giải pháp sử dụng *reference activation* (từ reference input) làm mốc so sánh.
 - *Cơ chế*: Tính contribution score dựa trên độ chênh lệch giữa activation hiện tại và tham chiếu thay vì dùng đạo hàm tại một điểm.
 - Có thể hiểu Gradient hỏi: Nếu input thay đổi rất nhỏ thì output thay đổi bao nhiêu? còn DeepLIFT hỏi: So với trạng thái reference, input hiện tại đã làm output thay đổi bao nhiêu?
 - *Ưu điểm*: Vẫn phát hiện được tầm quan trọng của feature ngay cả khi gradient triệt tiêu.

3 Ví dụ - Method

Xét network: $x \rightarrow h \rightarrow y$, trong đó $h = 2x$ và $y = 3h$

Chọn reference input: $x^0 = 0$

Chọn current input: $x = 2$

3.1 Forward Pass

Current activations: $h = 2x = 2 \times 1 = 2$; $y = 3h = 3 \times 2 = 6$

Reference activations: $h^0 = 2x^0 = 2 \times 0 = 0$; $y^0 = 3h^0 = 0$

Difference-from-reference:

- *Input*: $\delta_x = x - x^0 = 1 - 0 = 1$
- *Hidden*: $\delta_h = h - h^0 = 2 - 0 = 2$
- *Output*: $\delta_y = y - y^0 = 6 - 0 = 6$

Ý nghĩa: $\delta_y = 6$ nghĩa là output hiện tại lớn hơn reference output một lượng bằng 6.

Contribution Scores DeepLIFT định nghĩa C_{xy} là contribution từ neuron x tới neuron y .

- Summation-to-delta Property: DeepLIFT yêu cầu:

$$\sum_{i \in S} C_{x_i y} = \delta_y.$$

Ý nghĩa: Tổng contribution của tất cả input neurons phải bằng đúng phần output thay đổi, tức là:

$$\text{total contribution} = \text{actual output change}$$

Multipliers DeepLIFT định nghĩa: $C_{xy} = m_{xy} \delta_x$. Trong đó, δ_x là lượng thay đổi của input và m_{xy} là mức ảnh hưởng của x tới y .

- Linear Composition: Multiplier được propagate qua network theo công thức:

$$m_{xt} = \sum_{y \in O_x} m_{xy} m_{yt}$$

Lưu ý: Ý tưởng này tương tự chain rule ($\frac{\partial t}{\partial x} = \frac{\partial y}{\partial x} \frac{\partial t}{\partial y}$), nhưng DeepLIFT propagate contribution thay vì gradient.

Xét lại network: $x \xrightarrow{m_{xh}} h \xrightarrow{m_{hy}} y$ với $h = 2x$ và $y = 3h$.

Bước 1: Tính Multipliers từng chặng

- $m_{xh} = \frac{\delta_h}{\delta_x} = \frac{h-h^0}{x-x^0} = \frac{2-0}{1-0} = 2$
- $m_{hy} = \frac{\delta_y}{\delta_h} = \frac{y-y^0}{h-h^0} = \frac{6-0}{2-0} = 3$

Bước 2: Propagate (Linear Composition) Áp dụng công thức tương tự chain rule: $m_{xy} = m_{xh} \cdot m_{hy} = 2 \times 3 = 6$.

Bước 3: Kiểm chứng kết quả Dựa vào định nghĩa $C_{xy} = m_{xy} \delta_x$
 $C_{xy} = 6 \times 1 = 6$.

Kết quả: $C_{xy} = \delta_y = 6$, thỏa mãn tính chất Summation-to-delta.

Tại sao cần Multiplier? Trong ví dụ tuyến tính này, m_{xy} giống như một "hệ số tỉ lệ" giữa sự thay đổi đầu vào và sự thay đổi đầu ra trên một khoảng xác định (từ reference đến current).

Chain rule của DeepLIFT: Thay vì nhân các đạo hàm tại một điểm ($f'(x)$), thì nhân các hệ số tỉ lệ này lại với nhau. Điều này giúp thông tin về sự thay đổi không bị "mất dấu" khi đi qua các hàm kích hoạt bị bão hòa (như Sigmoid) hoặc bị triệt tiêu (như ReLU).

DeepLIFT propagate contribution thay vì gradient.

3.2 Backward Pass

Để tính toán hiệu quả trên mạng lớn, DeepLIFT thực hiện lan truyền ngược các Multipliers:

1. Khởi tạo: Gán $m_{yy} = 1$ tại lớp output.

2. Lan truyền ngược (Backpropagate): Nhân ngược các multiplier m đã lưu từ bước Forward để tìm ảnh hưởng tổng quát từ input x đến y :

$$m_{xy} = m_{xh} \cdot m_{hy} = 2 \times 3 = 6$$

(Tương tự chain rule: $\frac{\delta y}{\delta x} = \frac{\delta h}{\delta x} \cdot \frac{\delta y}{\delta h}$)

3. Kết quả Contribution Score: $C_{xy} = m_{xy} \cdot \delta_x = 6 \times 1 = 6$.

Lưu ý: Backward chỉ thực hiện **một lần duy nhất** cho toàn bộ các inputs, giúp tiết kiệm tài nguyên so với việc tính toán riêng lẻ từng contribution ở chiều thuận.

4 RevealCancel Rule

Vấn đề: Trong các hệ thống phi tuyến, các tín hiệu đóng góp có thể triệt tiêu lẫn nhau (cancel out). Ví dụ, trong hàm $y = \max(x_1, x_2)$, nếu một input tăng và một input giảm nhưng giá trị cực đại không đổi, các phương pháp dựa trên Gradient sẽ cho rằng cả hai không quan trọng (gradient = 0).

Giải pháp từ Paper: DeepLIFT tách biệt dòng tín hiệu dựa trên dấu của sự thay đổi.

- **Cơ chế tách luồng:** Tách sự thay đổi của input thành hai phần: dương (Δx^+) và âm (Δx^-).
- **Lan truyền độc lập:** Mỗi luồng được gán một multiplier riêng ($m_{\Delta x^+ \Delta y}$ và $m_{\Delta x^- \Delta y}$) để lan truyền về phía trước.
- **Hiệu quả:** Ngay cả khi tổng $\Delta y = 0$, việc tính toán riêng biệt giúp "tiết lộ" (reveal) rằng các feature vẫn có đóng góp lớn nhưng ngược dấu nhau, giúp phát hiện chính xác các *feature interactions*.

Giả sử ta có hàm: $y = \max(x_1, x_2)$.

1. Thiết lập giá trị:

- **Reference:** $x_1^0 = 0, x_2^0 = 0 \rightarrow y^0 = \max(0, 0) = 0$.
- **Current:** $x_1 = 10, x_2 = 5 \rightarrow y = \max(10, 5) = 10$.

$\Rightarrow$ Tổng thay đổi thực tế: $\delta_y = y - y^0 = 10 - 0 = 10$.

2. Xét một biến động (Perturbation): Giả sử từ trạng thái Current, x_1 giảm đi 2 đơn vị và x_2 tăng thêm 2 đơn vị:

- $x_1^{new} = 8, x_2^{new} = 7$.
- $y^{new} = \max(8, 7) = 8$.

Lúc này, δ_y mới là $8 - 10 = -2$. So với trạng thái trước ($y = 10$), output giảm 2 đơn vị.

3. Vấn đề của Gradient truyền thông: Nếu ta dùng đạo hàm tại điểm $x_1 = 10, x_2 = 5$, vì x_1 đang là giá trị lớn nhất nên $\frac{\partial y}{\partial x_1} = 1$ và $\frac{\partial y}{\partial x_2} = 0$. $\rightarrow$ Gradient bảo rằng x_2 không có đóng góp gì cả, dù x_2 đã tăng lên rất gần x_1 . Nếu x_2 tăng thêm chút nữa, nó sẽ chiếm quyền kiểm soát output.

4. Cách RevealCancel xử lý: DeepLIFT nhận thấy có sự triệt tiêu (Cancel):

- $\delta_{x_1} = 8 - 10 = -2$ (phần âm).
- $\delta_{x_2} = 7 - 5 = +2$ (phần dương).

Thay vì chỉ nhìn vào kết quả tổng δ_y giảm 2, RevealCancel tách riêng:

- Nó ghi nhận x_2 đã có một "nỗ lực" tăng lên (+2) để giành quyền kiểm soát output.
- Nó ghi nhận x_1 đã sụt giảm (-2).

Kết quả: Thay vì gán toàn bộ sự sụt giảm -2 cho x_1 (như Gradient), RevealCancel sẽ phân bổ điểm số (contribution) cho cả hai: x_2 nhận điểm dương vì nó đã cố gắng kéo output lên, và x_1 nhận điểm âm vì nó kéo output xuống. Điều này "tiết lộ" (Reveal) tầm quan trọng của x_2 mà Gradient đã bỏ lỡ.

Version cũ (Rescale Rule): Sử dụng một Multiplier duy nhất $m = \frac{\Delta y}{\Delta x}$.

$$C_x = m \cdot \Delta x$$

Hạn chế: Nếu $\Delta x = 0$ (do các thành phần bên trong triệt tiêu nhau), toàn bộ đóng góp bị xóa sổ ($C_x = 0$), gây ra hiện tượng *Incomplete Explanation*.

Version 2017 (RevealCancel Rule): Tách biệt sự thay đổi thành luồng dương (+) và âm (-). Công thức lan truyền Multiplier trở thành:

$$\Delta y^+ = m_{\Delta x^+ \Delta y^+} \Delta x^+ + m_{\Delta x^- \Delta y^+} \Delta x^-$$

$$\Delta y^- = m_{\Delta x^+ \Delta y^-} \Delta x^+ + m_{\Delta x^- \Delta y^-} \Delta x^-$$

Điểm mới cốt lõi:

- **Không triệt tiêu:** Các tín hiệu đối lập được giữ riêng biệt trong suốt quá trình Backward.
- **Phát hiện Dependencies:** RevealCancel có khả năng giải thích các hàm logic phức tạp (như XOR hoặc cổng logic) nơi mà sự hiện diện của một feature này có thể làm lu mờ feature khác.

5 Final Activation & Saturation Problem

Vấn đề Saturation (Bão hòa): Ở lớp cuối (Softmax hoặc Sigmoid), khi input đã rất lớn, output bị chặn ở ngưỡng 1.0. Điều này làm triệt tiêu gradient và gây ra hiện tượng *attenuation* (suy giảm công trạng), khiến các feature quan trọng bị đánh giá thấp hơn thực tế.

Ví dụ minh họa ($y = \sigma(x_1 + x_2)$):

- Nếu $x_1 = 100, x_2 = 0 \rightarrow x_1$ chiếm 100% công trạng vì mình nó đủ đẩy output lên 1.
- Nếu cả $x_1 = 100, x_2 = 100 \rightarrow$ Output vẫn chỉ là 1, công trạng bị chia đôi mỗi bên 0.5 dù bản chất mỗi feature đều cực kỳ mạnh.

Giải pháp từ Paper (Linearization):

- **Tính toán trên Logits:** Thay vì phân bổ sự thay đổi của xác suất cuối cùng (P), DeepLIFT thực hiện phân bổ trên giá trị **Pre-activation scores** (z - trước khi đi qua hàm kích hoạt).
- **Lý do:** Giá trị logit $z = Wx + b$ là một hàm tuyến tính, không bị giới hạn trên và không bị bão hòa.
- **Kết quả:** Giữ nguyên được độ nhạy của tín hiệu và phản ánh đúng giá trị thực của các feature đầu vào mà không bị ảnh hưởng bởi tính chất "thắt nút" của hàm Softmax/Sigmoid.

6 Kết quả thực nghiệm (Results)

Paper đánh giá DeepLIFT trên: MNIST và Sinh học phân tử (Genomics).

6.1 MNIST

- **Thử nghiệm:** Sử dụng kỹ thuật "nhân tính" (shuffling) hoặc xóa bỏ các pixel mà DeepLIFT đánh giá là quan trọng nhất.
- **Kết quả:** Khi xóa các pixel có điểm contribution cao theo DeepLIFT, độ chính xác của mô hình giảm nhanh hơn đáng kể so với khi xóa dựa trên các phương pháp Gradient khác.
- **Phát hiện:** DeepLIFT loại bỏ nhiễu tốt hơn, giúp bản đồ nhiệt (saliency maps) tập trung đúng vào hình dáng của con số thay vì các pixel nền ngẫu nhiên.

6.2 Dữ liệu Genomics (Mô phỏng)

Đây là phần DeepLIFT thể hiện sức mạnh vượt trội nhất nhờ quy tắc *RevealCancel*.

- **Bài toán:** Phát hiện các motif DNA (chuỗi quy định) trong các trình tự gen phức tạp.
- **Vấn đề của Gradient:** Các phương pháp cũ thường gặp hiện tượng "bão hòa" khi một motif xuất hiện quá nhiều lần, dẫn đến việc đánh giá thấp tầm quan trọng của chúng.
- **Kết quả của DeepLIFT:**
 - Khôi phục được các motif bị che khuất nhờ cơ chế tách luồng đóng góp dương và âm.
 - Độ chính xác trong việc xác định vị trí các đặc trưng quan trọng cao hơn 20-30

6.3 Hiệu năng tính toán

Một kết quả quan trọng khác là tốc độ:

- DeepLIFT chỉ mất **một lần Backward duy nhất** để tính toán toàn bộ contribution.
- So với Integrated Gradients (cần 20-100 lần chạy để xấp xỉ tích phân), DeepLIFT nhanh hơn hàng chục lần nhưng vẫn cho kết quả ổn định và ít nhiễu hơn.

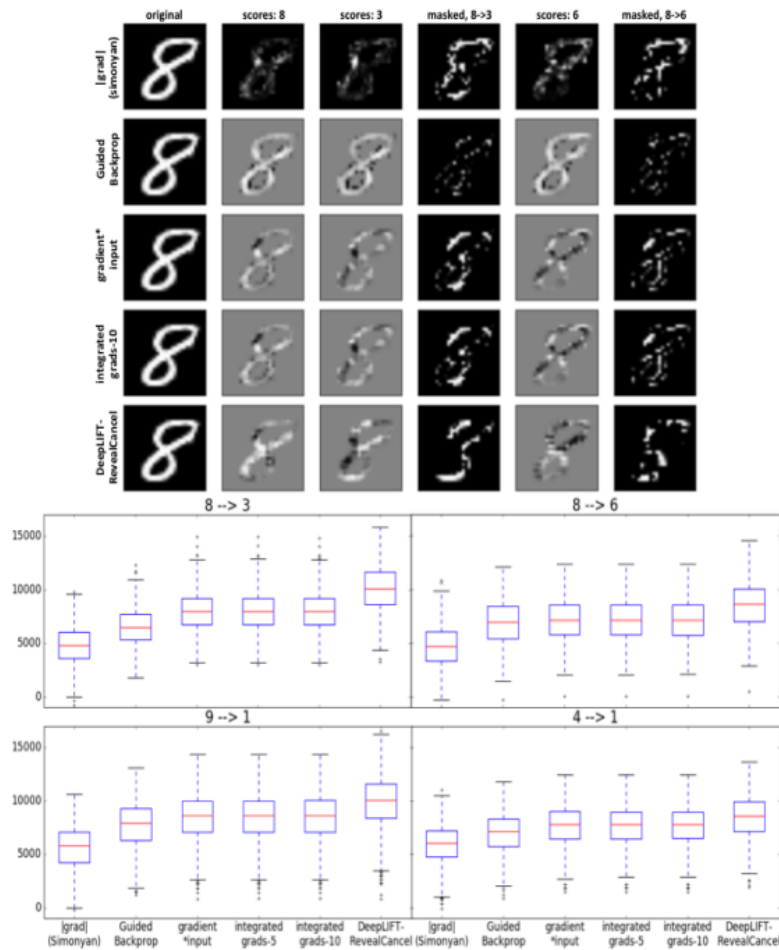


Figure 4: Hiệu quả trên dữ liệu hình ảnh (MNIST)

Nội dung: So sánh khả năng xác định các pixel quan trọng để chuyển đổi dự đoán từ chữ số này sang chữ số khác (ví dụ: từ số 8 sang số 3 hoặc số 6).

- **Thử nghiệm:** Masking (che đi) các pixel được đánh giá là quan trọng nhất.
- **Kết quả:** DeepLIFT với RevealCancel dẫn đến sự thay đổi *log-odds* (tỷ lệ logarit của xác suất) cao nhất khi chuyển đổi nhãn ($8 \rightarrow 6$).
- **Biểu đồ Boxplot:** Cho thấy trên 1,000 ảnh thử nghiệm, DeepLIFT giúp tăng xác suất của nhãn mục tiêu (target class) hiệu quả hơn hẳn so với Integrated Gradients hay Gradient*Input.

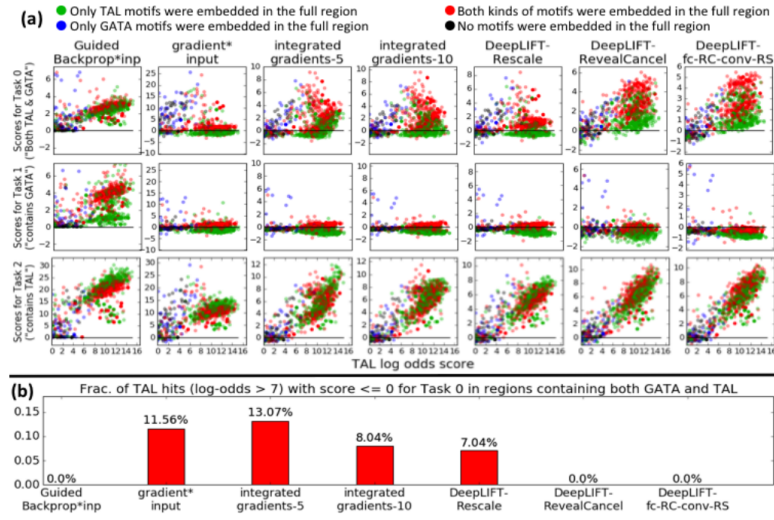


Figure 5: Hiệu quả trên dữ liệu mô phỏng Genomic (TAL-GATA) Nội dung: Đánh giá mối tương quan giữa điểm quan trọng (importance score) và cường độ của motif TAL1.

- **Scatter plots:** Biểu đồ phân tán giữa điểm quan trọng và log-odds của motif match.
- **Phát hiện:** DeepLIFT (kết hợp RevealCancel ở lớp fully-connected và Rescale ở lớp conv) giúp giảm nhiễu đáng kể.
- **False Negatives:** DeepLIFT với RevealCancel không có trường hợp âm tính giả (không bỏ sót các motif mạnh), vượt trội hơn các phương pháp khác vốn thường bỏ sót motif do hiện tượng triệt tiêu tín hiệu.

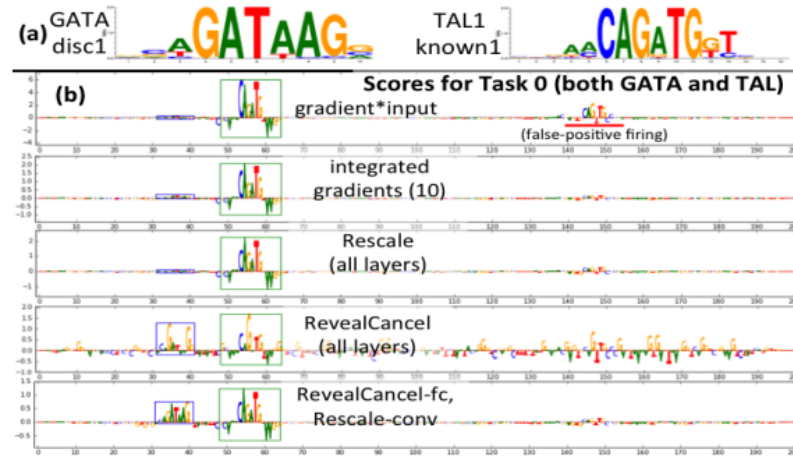


Figure 6: Khám phá Motif thực tế (Task 0) Nội dung: Minh họa khả năng làm nổi bật các motif GATA1 và TAL1 trong một chuỗi DNA cụ thể.

- **Visualization:** Sử dụng biểu đồ chữ (letter height phản ánh điểm số).
- **Đặc điểm:** RevealCancel làm nổi bật cả hai motif GATA1 (hộp xanh dương) và TAL1 (hộp xanh lá).
- **Độ chính xác:** Nó phân biệt được motif TAL1 thực sự và các motif giả (ngẫu nhiên) xuất hiện trong chuỗi (gạch chân đỏ), điều mà các phương pháp cũ thường nhầm lẫn hoặc đánh giá sai lệch.

NOTE: rõ hơn thí nghiệm

Load model và ảnh số 8.

Khi bạn đưa ảnh số 8 vào, lớp cuối cùng của mô hình (Softmax hoặc Logits) sẽ có 10 node đầu ra (từ 0 đến 9).

Bình thường, node số 8 sẽ có giá trị rất cao (ví dụ: 15.0).

Node số 6 sẽ có giá trị rất thấp (ví dụ: -2.0).

→ lấy y output là 6 rồi tính lại multiplier ...